Mini Help Desk

Sistema de Gestão de Chamados de Suporte

Documentação técnica - Etapa 3

Python 3 • Programação Orientada a Objetos

Repositório do código: https://github.com/BrunoFMalta/ProjetoPythonPOO

1. Apresentação do projeto

O Mini Help Desk é um sistema de gestão de chamados de suporte de TI desenvolvido 100% em Python orientado a objetos, com interface em terminal. Ele organiza o ciclo de vida de um chamado: abertura pelo cliente, triagem de prioridade, atribuição a um técnico, escalonamento entre níveis de suporte, resolução e fechamento, com controle de prazos (SLA) e histórico de todas as interações.

Problema e público

Equipes pequenas de suporte costumam controlar chamados em planilhas ou conversas de mensagens, sem prioridade definida por critério, sem histórico confiável e sem visibilidade de prazos estourados. O sistema substitui esse controle informal por regras explícitas e relatórios gerenciais. O público-alvo são coordenadores e técnicos de equipes de suporte de pequeno porte.

O modelo de domínio foi inspirado em ferramentas de gestão de serviços de TI, como o GLPI (tipos de chamado, ciclo de status, SLA e prioridade calculada por urgência e impacto), em uma versão enxuta, offline e de finalidade didática.

Níveis de suporte

- **N1:** atendimento por telefone, resolve problemas simples.
- **N2:** atendimento presencial, resolve problemas mais complexos e os que o N1 não consegue resolver (exemplo: troca de ponteira RJ45 de um cabo de rede).

Funcionalidades

- Abrir chamado do tipo Incidente ou Requisição, com prioridade calculada automaticamente.
- Listar chamados, com prioridade, nível, status, técnico responsável e prazo.
- Atribuir chamado a um técnico, respeitando o nível de atendimento.
- Escalar chamado de N1 para N2, devolvendo-o à fila sem técnico.
- Resolver e fechar chamados, com validação do ciclo de status.
- Consultar detalhes e histórico completo de um chamado.
- Relatórios: chamados com SLA estourado, carga por técnico e tempo médio de resolução.

Entradas e saídas

A entrada de dados é feita pelo teclado, por meio de menus numerados e formulários em texto. A saída é exibida no terminal, em listas e tabelas de texto. Nesta etapa os dados ficam apenas em memória (sem leitura nem gravação de arquivos).

2. Requisitos da disciplina atendidos

Requisito	Como foi atendido	Arquivo
Mínimo de 5 classes	10 classes: `Usuario`, `Cliente`, `Tecnico`, `Chamado`, `Incidente`, `Requisicao`, `Interacao`, `CalendarioComercial`, `Central`, `Relatorio` (mais 6 enumerações).	usuarios.py, chamados.py, calendario.py, central.py
Herança	`Cliente` e `Tecnico` herdam de `Usuario`; `Incidente` e `Requisicao` herdam de `Chamado`.	usuarios.py, chamados.py
Polimorfismo	`calcular_prazo()`: o incidente calcula o prazo pela prioridade; a requisição usa prazo fixo. `descricao()`: cada tipo de usuário se descreve de um jeito.	chamados.py, usuarios.py
Associação	`Chamado` se associa a um `Cliente` (solicitante), a no máximo um `Tecnico` (atendente) e a várias `Interacao` (histórico).	chamados.py
Método privado	`__calcular_prioridade()` e `__transicao_valida()` em `Chamado`.	chamados.py
Atributo privado	`__status`, `__nivel_atual` e `__prioridade` em `Chamado`, lidos por `properties`.	chamados.py
Programa principal cria 2+ objetos	A função `criar_dados_exemplo()` instancia `Cliente`, `Tecnico` e `Incidente`, e `main()` instancia `Central` e `CalendarioComercial`.	main.py

3. Estrutura do projeto

Arquivo	Responsabilidade
`enums.py`	Enumerações: Status, Urgência, Impacto, Prioridade, Nível (N1/N2) e Tipo de Interação.
`usuarios.py`	Classe abstrata `Usuario` e as subclasses `Cliente` e `Tecnico`.
`chamados.py`	Classe abstrata `Chamado` (regras de negócio), subclasses `Incidente` e `Requisicao`, e a classe `Interacao`.
`calendario.py`	Classe `CalendarioComercial`: cálculo de prazos em horas úteis.
`central.py`	Classes `Central` (guarda os dados em memória) e `Relatorio` (relatórios gerenciais).
`main.py`	Programa principal: dados de exemplo, menu e interação com o usuário.
`test_helpdesk.py`	Testes automatizados com pytest.

Separação de responsabilidades: as regras de negócio ficam nas classes de domínio; o `main.py` apenas lê o teclado, chama os métodos das classes e imprime o resultado. Por isso a interface pode ser trocada no futuro sem alterar as regras.

Execução: `python main.py` inicia o sistema (Python 3.10 ou superior) e `pytest` executa os testes.

4. Descrição das classes

Legenda de visibilidade: **+** público; **-** privado (no código, o nome começa com __, com name mangling); **#** restrito (no código, o nome começa com _). Nesta seção os nomes aparecem sem esses prefixos.

Usuario (abstrata) - usuarios.py

Base de qualquer pessoa cadastrada no sistema.

- +id, +nome, +email.
- +descricao(): método abstrato, texto curto usado nas listas (polimórfico).

Cliente - usuarios.py

Quem abre chamados.

- +setor: setor onde trabalha.
- +descricao(): devolve nome e setor.

Tecnico - usuarios.py

Quem atende chamados. O nível (N1 ou N2) limita o que pode atender.

- +nivel: N1 ou N2; #lista interna de chamados atribuídos (acesso externo por `chamados`, que devolve uma cópia).
- +pode_atender(chamado): N2 atende qualquer chamado; N1 só os de nível N1.
- +carga_atual(): quantidade de chamados ainda em andamento com o técnico.
- #receber(chamado) e #liberar(chamado): uso restrito; só `Chamado.atribuir()` e `Chamado.escalar()` os chamam, mantendo as duas pontas da associação sincronizadas.
- +descricao(): devolve nome e nível.

Chamado (abstrata) - chamados.py

Centro do sistema. Concentra as regras de prioridade, ciclo de status, atribuição, escalonamento e prazo.

- +id, +titulo, +descricao, +solicitante, +urgencia, +impacto, +data_abertura, +data_resolucao, +tecnico.
- -status, -nivel_atual, -prioridade: privados; lidos pelas properties `status`, `nivel_atual` e `prioridade`.
- +atribuir(tecnico): valida status e nível do técnico, registra a atribuição e passa o chamado para Em atendimento.
- +mudar_status(novo, autor): só aceita transições permitidas; caso contrário levanta `ValueError`.
- +escalar(autor, motivo): N1 devolve o chamado para a fila do N2, sem técnico, registrando o motivo.
- +registrar_interacao(autor, texto, tipo): acrescenta um registro ao histórico.
- +sla_estourado(agora, cal): indica se o prazo já foi ultrapassado.
- +calcular_prazo(cal): método abstrato, implementado por cada subtipo.
- -calcular_prioridade() e -transicao_valida(novo): regras internas privadas.

Incidente - chamados.py

Chamado de algo que parou de funcionar. O prazo depende da prioridade.

- +servico_afetado.
- +calcular_prazo(cal): usa a tabela de horas úteis por prioridade (seção 5).

Requisicao - chamados.py

Chamado de pedido de algo novo. O prazo é fixo, independente da prioridade.

- +item_solicitado.
- +calcular_prazo(cal): 40 horas úteis a partir da abertura.

Interacao - chamados.py

Um registro do histórico do chamado (comentário, mudança de status, atribuição, escalonamento ou solução).

- +autor, +texto, +tipo, +data.
- +__str__(): linha formatada para exibição.

CalendarioComercial - calendario.py

Regra de tempo: conta somente horas úteis (segunda a sexta, das 8h às 18h, sem feriados).

- -hora_inicio, -hora_fim, -dias_uteis.
- +somar_horas_uteis(inicio, horas): devolve o momento em que as horas úteis terminam, pulando noites e fins de semana.
- +horas_uteis_entre(inicio, fim): mede o intervalo em horas úteis.
- -eh_dia_util(), -eh_expediente(), -proximo_expediente(): auxiliares privados.

Central - central.py

Ponto único de acesso aos dados em memória; coordena a abertura e a busca de chamados.

- +calendario; #listas internas de chamados e usuários.
- +cadastrar_usuario(), +abrir_chamado(), +atribuir(), +buscar(status, tecnico, nivel), +chamados_do_tecnico(), +clientes(), +tecnicos().
- -proximo_id(colecao): gera identificadores sequenciais.

Relatorio - central.py

Relatórios gerenciais calculados sobre os dados da `Central`.

- +sla_estourados(agora): chamados que passaram do prazo.
- +carga_por_tecnico(): chamados em andamento por técnico.
- +tempo_medio_resolucao(): média, em horas úteis, entre abertura e resolução.

5. Regras de negócio

Prioridade calculada

A prioridade não é digitada: é calculada na criação do chamado, somando o peso da urgência (1 a 3) e o do impacto (1 a 3).

Soma (urgência + impacto)	Prioridade
2 ou 3	BAIXA
4	MEDIA
5	ALTA
6	CRITICA

Ciclo de status

O status é privado e só muda por `mudar_status()`, que valida a transição. Tentativas fora da tabela (por exemplo, fechar um chamado ainda aberto) levantam `ValueError`, tratada pelo menu.

Status atual	Transições permitidas
Aberto	Em atendimento
Em atendimento	Pendente, Solucionado
Pendente	Em atendimento
Solucionado	Fechado, Em atendimento (reabertura)
Fechado	nenhuma

Escalonamento N1 para N2

Todo chamado nasce no nível N1. Se o técnico N1 não consegue resolver (por exemplo, por exigir atendimento presencial), ele chama `escalar()`: o chamado passa para o nível N2, perde o técnico, volta ao status Aberto (na fila do N2) e o motivo é gravado no histórico. Depois disso, um técnico N1 não consegue mais pegar o chamado; um N2 consegue. Técnicos N2 podem atender também chamados de nível N1.

Prazos (SLA) em horas comerciais

Os prazos contam apenas horas úteis: segunda a sexta, das 8h às 18h. Um chamado aberto na sexta às 17h com prazo de 2 horas úteis vence na segunda às 9h.

Tipo	Prioridade	Prazo (horas úteis)
Incidente	CRITICA	4
Incidente	ALTA	8
Incidente	MEDIA	16
Incidente	BAIXA	24
Requisição	qualquer	40 (5 dias úteis)

6. Testes automatizados

O arquivo `test_helpdesk.py` reúne 9 testes com pytest, que verificam as regras de negócio principais:

- Prioridade calculada a partir de urgência e impacto.
- Transição de status inválida é barrada e o status se mantém.
- Escalonamento N1 para N2: nível, técnico, status, histórico e bloqueio do N1.
- Fluxo completo: atribuir, solucionar e fechar, com data de resolução.
- Calendário comercial pulando noites e fins de semana.
- Cálculo de horas úteis entre dois momentos.
- SLA estourado e dentro do prazo.
- Polimorfismo do prazo: incidente e requisição calculam prazos diferentes.
- Relatório de carga por técnico.

7. Uso de CRUD e bibliotecas

CRUD de chamados

- **Create:** abertura de chamados pelo menu.
- **Read:** listagem, detalhe com histórico e relatórios.
- **Update:** atribuição, escalonamento e mudanças de status.
- **Delete:** não implementado, de propósito: chamados não são apagados; o fechamento preserva o histórico para auditoria.

Bibliotecas

O sistema usa somente a biblioteca padrão do Python: `datetime` (datas e prazos), `enum` (enumerações), `abc` (classes abstratas) e `typing` (anotações de tipo). A única dependência externa é o `pytest`, usado apenas nos testes.

8. Limitações conhecidas e próximos passos

Limitação conhecida (estado atual)	Melhoria prevista para a entrega final
Os dados ficam só em memória: ao fechar o programa, tudo é reiniciado.	Persistência em arquivo (JSON ou SQLite).
Usuários são criados no código; o menu não cadastra clientes nem técnicos.	Cadastro de usuários pelo menu.
O calendário não considera feriados e o expediente é fixo (seg a sex, 8h às 18h).	Expediente configurável e lista de feriados.
Interface apenas em terminal, em texto simples.	Refinamento visual da interface.
A prioridade não é recalculada se urgência ou impacto mudarem depois da abertura (o menu não permite editar esses campos).	Edição de chamado com recálculo de prioridade.